

A Regular Expression (**Regex**) module is used for finding, matching, replacing, or validating text patterns.

Think of it as a powerful text-search tool.

Example:

Text:

```
Hello!!! Welcome@@@
```

You want only letters and numbers.

Using:

```
re.sub(r'^a-z0-9 ', '', text)
```

Result:

```
hello welcome
```

So in your project:

```
import re
```

is needed because:

```
text = re.sub(r'^a-z0-9 ', '', text)
```

uses the **re** module to remove unwanted characters before sending the email to the ML model.

◆ import re

```
import re
```

import → keyword

re → Regular Expression module

👉 Use: text cleaning and pattern matching

◆ CLEAN TEXT FUNCTION

```
def clean_text(text):
```

def → keyword



clean_text → function name

text → parameter

👉 Use: define a function that cleans email text before prediction



◆ CONVERT TEXT TO LOWERCASE

```
text = text.lower()
```

text → variable

= → assignment operator

lower() → string method

👉 Use: convert all letters into lowercase

Example:

```
FREE MONEY
```

becomes:

```
free money
```

This helps the model treat words consistently.

◆ REMOVE SPECIAL CHARACTERS

```
text = re.sub(r'^a-z0-9 ', '', text) Check sub pdf
```

text → variable

re → Regular Expression module

sub() → substitute method

r'^a-z0-9 ' → pattern

" → replace with nothing

👉 Use: remove special characters from text

Example:

Input:

```
Win $$$ Now!!!
```

Output:

```
win now
```

Characters like:

```
$
!
@
#
%
```

&
*

are removed.

◆ RETURN CLEANED TEXT

```
return text
```

return → keyword

text → cleaned text

👉 Use: send cleaned text back to the caller

Example:

```
clean_text("HELLO!!!")
```

returns:

```
hello
```

◆ FORCE JSON READING

```
data = request.get_json(force=True) //check data
```

data → variable

request → Flask request object

get_json() → method

force=True → parameter

👉 Use: force Flask to read incoming request data as JSON

The JSON received from Spring Boot is stored inside:

```
data
```

◆ EXTRACT EMAIL TEXT

```
email_text = data.get("emailText")
```

email_text → variable

data → dictionary 

get() → dictionary method

"emailText" → key

👉 Use: retrieve the email text from incoming JSON

Example:

Input JSON:

```
{
  "emailText": "You won a lottery"
}
```

Result:

```
email_text = "You won a lottery"
```

◆ VALIDATE INPUT

```
if not email_text:
```

if → keyword

not → logical operator

email_text → variable

👉 Use: **check whether email text exists**

Examples that make condition **true**:

```
None
```

```
""
```

If the text is missing, Flask returns an error response.

◆ RETURN ERROR RESPONSE

```
return jsonify({"prediction": "ERROR"})
```

return → keyword

jsonify() → function

{ } → dictionary

"prediction" → key

"ERROR" → value

👉 Use: send error response when email text is missing

Response:

```
{
  "prediction": "ERROR"
}
```

◆ CLEAN THE EMAIL TEXT

```
email_text = clean_text(email_text)
```

email_text → variable

clean_text() → function

👉 Use: clean email text before sending it to the ML model

Flow:

```
Original Text
  ↓
clean_text()
  ↓
Lowercase
  ↓
Remove Special Characters
  ↓
Clean Text
```

◆ CONVERT TEXT TO VECTOR

```
vector = vectorizer.transform([email_text])
```

vector → variable

vectorizer → CountVectorizer object

transform() → method // is a method of the vectorizer object.

[email_text] → list

👉 Use: convert text into numerical features

Machine learning models cannot understand raw text.

Example:

```
free money now
```

becomes a numerical vector.

◆ MAKE PREDICTION

```
prediction = model.predict(vector)[0] // check predict pdf
```

prediction → variable

model → trained ML model

predict() → method

vector → input data

[0] → index operator

👉 Use: get prediction result from the model

Example:

```
[1]
```

becomes:

```
1
```

stored in prediction.

◆ ASSIGN RESULT

```
result = "SPAM" if prediction == 1 else "HAM"
```

result → variable

"SPAM" → spam label

"HAM" → non-spam label

if → conditional operator

else → alternate value

👉 Use: convert model output into readable text

If:

```
prediction == 1
```

Result:

```
SPAM
```

Otherwise:

```
HAM
```

◆ RETURN JSON RESPONSE

```
return jsonify({"prediction": result})
```

return → keyword

jsonify() → function

{ } → dictionary

"prediction" → key

result → value

👉 Use: send prediction result back to Spring Boot

Example:

```
{
  "prediction": "SPAM"
}
```

or

```
{
  "prediction": "HAM"
}
```

◆ START FLASK SERVER

```
app.run(host="0.0.0.0", port=5000)
```

app → Flask application object

run() → method

host → parameter

"0.0.0.0" → IP address

port → parameter

5000 → port number

👉 Use: start Flask server

The API becomes available on port:

```
5000
```

and can accept requests from network interfaces.

🔥 FINAL FLOW

```
Input JSON
  ↓
Extract Email Text
  ↓
Validate Input
  ↓
Clean Text
  ↓
Convert Text To Vector
  ↓
ML Model Prediction
  ↓
SPAM or HAM
  ↓
JSON Response
```